

SOLID Principles – Designing Enterprise-Level Systems

1. From Classes to Enterprise Systems

So far, we have learned how to create classes such as:

```
class Account:
    def deposit(self, amount):
        ...

    def withdraw(self, amount):
        ...
```

This works well when the application is small.

But imagine that our banking system grows.

We now need:

- Savings Accounts
- Current Accounts
- Salary Accounts
- Fixed Deposits
- Loan Accounts
- Different withdrawal rules
- Different interest calculations
- PIN authentication
- OTP authentication
- Transaction logging
- Email notifications
- SMS notifications
- Database storage
- Fraud detection
- Audit trails
- Different payment methods

If we continue putting everything inside one or two classes, the code becomes difficult to maintain.

For example:

```
class Bank:
    def create_account(self):
        ...

    def calculate_interest(self):
        ...

    def send_email(self):
        ...

    def send_sms(self):
        ...

    def validate_pin(self):
        ...

    def save_to_database(self):
        ...

    def generate_report(self):
        ...

    def detect_fraud(self):
        ...
```

This class is becoming responsible for **too many things**.

A small change in one feature can break another feature.

This is where **SOLID principles** become important.

2. What is SOLID?

SOLID is a collection of five object-oriented design principles that help us build software that is:

- Maintainable
- Flexible
- Testable
- Reusable

- Extensible
- Less tightly coupled
- Easier to understand

SOLID stands for:

Letter	Principle	Main Idea
S	Single Responsibility Principle	One class should have one responsibility
O	Open/Closed Principle	Open for extension, closed for modification
L	Liskov Substitution Principle	Child classes must be safely substitutable for parent classes
I	Interface Segregation Principle	Don't force classes to depend on methods they don't need
D	Dependency Inversion Principle	Depend on abstractions, not concrete implementations

A useful way to remember them:

S – One job **O** – Add without breaking **L** – Child should behave like parent **I** – Keep interfaces small **D** – Depend on contracts

3. Why SOLID Matters in Enterprise Applications

Consider a banking application used by millions of customers.

Suppose the business says:

"We are changing our notification provider from Email to WhatsApp."

In a poorly designed system, we might have:

```
class Account:
    def withdraw(self, amount):
        ...

        send_email(...)
```

Now changing notification systems requires modifying `Account` .

But `Account` should not care **how notifications are sent**.

Its responsibility is managing account operations.

A better architecture separates:

```
Account
|
+---- Transaction Service
|
+---- Notification Service
|
+---- Authentication Service
|
+---- Repository
```

Each component has a specific responsibility.

That is the thinking behind SOLID.

S — Single Responsibility Principle

4. What is SRP?

Single Responsibility Principle:

A class should have one reason to change.

This does **not** necessarily mean:

"A class should contain only one method."

Instead, it means that a class should represent **one cohesive responsibility**.

5. Bad Example – Multiple Responsibilities

Consider:

```
class BankAccount:
    def deposit(self, amount):
        self.balance += amount

    def withdraw(self, amount):
        self.balance -= amount

    def save_to_database(self):
        ...

    def send_email(self):
        ...

    def generate_statement(self):
        ...

    def calculate_tax(self):
        ...
```

What responsibilities does this class have?

It handles:

- Account operations
- Database persistence
- Email communication
- Statement generation
- Tax calculation

Therefore, the class has many reasons to change.

For example:

Database requirements change

↓

BankAccount changes

Email provider changes

↓

BankAccount changes

Tax rules change

↓

BankAccount changes

Statement `format changes`

↓

BankAccount `changes`

This is a violation of SRP.

6. SRP-Compliant Design

Split the responsibilities.

```
class BankAccount:
    def deposit(self, amount):
        self.balance += amount

    def withdraw(self, amount):
        self.balance -= amount
```

Database responsibility:

```
class AccountRepository:
    def save(self, account):
        print("Saving account to database")
```

Notification responsibility:

```
class NotificationService:
    def send(self, message):
        print(f"Sending notification: {message}")
```

Statement responsibility:

```
class StatementGenerator:
    def generate(self, account):
        return f"Statement for {account.account_number}"
```

Now:

BankAccount

↓

`Account` operations

`AccountRepository`



`Database`

`NotificationService`



`Notifications`

`StatementGenerator`



`Statements`

Each class has a clear responsibility.

7. SRP in Our Banking System

Our Activity 3 `Account` class already contains several responsibilities:

```
deposit()  
withdraw()  
close_account()  
reopen_account()  
set_pin()  
verify_pin()
```

For a teaching-level application, this is acceptable.

But in a large enterprise system, we could eventually separate:

```
Account  
AuthenticationService  
TransactionService  
AccountLifecycleService  
AccountRepository  
NotificationService
```

The goal is not to blindly create hundreds of classes.

The goal is **high cohesion**.

0 – Open/Closed Principle

8. What is OCP?

Open/Closed Principle:

Software entities should be open for extension but closed for modification.

In simple terms:

We should be able to add new behavior without constantly modifying existing, tested code.

9. Bad Example – Growing if/elif Chains

Suppose our bank calculates interest.

```
class InterestCalculator:

    def calculate(self, account_type, balance):

        if account_type == "Savings":
            return balance * 0.04

        elif account_type == "Current":
            return balance * 0.01

        elif account_type == "Salary":
            return balance * 0.05
```

Initially this works.

But tomorrow the bank introduces:

```
Fixed Deposit
Recurring Deposit
NRI Account
Senior Citizen Account
Student Account
```

We keep modifying:


```
if account_type == "Savings":  
    ...  
  
elif account_type == "Current":  
    ...  
  
elif account_type == "Salary":  
    ...  
  
elif account_type == "Fixed Deposit":  
    ...  
  
elif account_type == "NRI":  
    ...
```

The class keeps changing.

This becomes difficult to maintain and test.

10. OCP Solution

Create an abstraction:

```
from abc import ABC, abstractmethod  
  
class InterestPolicy(ABC):  
  
    @abstractmethod  
    def calculate(self, balance: float) -> float:  
        pass
```

Savings:

```
class SavingsInterestPolicy(InterestPolicy):  
  
    def calculate(self, balance: float) -> float:  
        return balance * 0.04
```

Current:

```
class CurrentInterestPolicy(InterestPolicy):  
  
    def calculate(self, balance: float) -> float:  
        return balance * 0.01
```

Salary:

```
class SalaryInterestPolicy(InterestPolicy):  
  
    def calculate(self, balance: float) -> float:  
        return balance * 0.05
```

Now we can add:

```
class FixedDepositInterestPolicy(InterestPolicy):  
  
    def calculate(self, balance: float) -> float:  
        return balance * 0.07
```

without modifying the existing policies.

That is the essence of OCP.

11. Strategy Pattern Connection

Students should notice something important.

OCP frequently leads us toward the **Strategy Pattern**.

Instead of:

```
if account_type == "Savings":  
    ...  
elif account_type == "Current":  
    ...
```

we select a strategy:

```
policy = SavingsInterestPolicy()
```

and use:

```
policy.calculate(balance)
```

The algorithm can now change independently.

L – Liskov Substitution Principle

12. What is LSP?

Liskov Substitution Principle:

Objects of a child class should be usable wherever objects of the parent class are expected, without breaking the correctness of the program.

This is one of the most important—and commonly misunderstood—SOLID principles.

13. Simple Example

Suppose:

```
class Account:
    def withdraw(self, amount):
        ...
```

And:

```
class SavingsAccount(Account):
    def withdraw(self, amount):
        ...
```

This is reasonable if `SavingsAccount` behaves according to the contract established by `Account`.

But imagine:

```
class FixedDepositAccount(Account):  
  
    def withdraw(self, amount):  
        raise Exception("Withdrawal is impossible")
```

If the parent contract says:

```
account.withdraw(1000)
```

is a valid operation, then replacing:

```
Account
```

with:

```
FixedDepositAccount
```

breaks the program.

This may indicate that the abstraction is wrong.

14. Classic Rectangle/Square Example

Consider:

```
class Rectangle:  
  
    def set_width(self, width):  
        self.width = width  
  
    def set_height(self, height):  
        self.height = height  
  
    def area(self):  
        return self.width * self.height
```

Then:

```
class Square(Rectangle):  
  
    def set_width(self, width):  
        self.width = width  
        self.height = width  
  
    def set_height(self, height):  
        self.width = height  
        self.height = height
```

Now:

```
rectangle = Square()  
  
rectangle.set_width(10)  
rectangle.set_height(20)  
  
print(rectangle.area())
```

A normal rectangle would have:

```
10 × 20 = 200
```

But the square produces:

```
20 × 20 = 400
```

The child does not behave according to the expectations of the parent.

Therefore, inheritance is inappropriate here.

15. LSP in Banking

Suppose we create:

```
class WithdrawableAccount:  
    def withdraw(self, amount):
```

...

Savings Account:

```
class SavingsAccount(WithdrawableAccount):  
    def withdraw(self, amount):  
        ...
```

Current Account:

```
class CurrentAccount(WithdrawableAccount):  
    def withdraw(self, amount):  
        ...
```

Both satisfy the withdrawal contract.

But Fixed Deposit may not:

```
class FixedDepositAccount:  
    def withdraw(self, amount):  
        raise Exception("Cannot withdraw")
```

Therefore, it should not necessarily inherit from `WithdrawableAccount`.

Instead:

```
Account  
├── SavingsAccount  
├── CurrentAccount  
└── FixedDepositAccount
```

And only:

```
WithdrawableAccount  
├── SavingsAccount  
└── CurrentAccount
```

if that abstraction makes sense for the domain.

I — Interface Segregation Principle

16. What is ISP?

Interface Segregation Principle:

Clients should not be forced to depend on methods they do not use.

In Python, interfaces are commonly represented using:

```
ABC
```

and:

```
abstractmethod
```

17. Bad Interface

Suppose we create:

```
from abc import ABC, abstractmethod
```

```
class BankService(ABC):
```

```
    @abstractmethod
```

```
    def deposit(self, amount):  
        pass
```

```
    @abstractmethod
```

```
    def withdraw(self, amount):  
        pass
```

```
    @abstractmethod
```

```
    def transfer(self, amount):  
        pass
```

```
    @abstractmethod
```

```
    def print_statement(self):  
        pass
```

```
@abstractmethod
def apply_for_loan(self):
    pass
```

Now imagine:

```
class ATM(BankService):
    ...
```

Does an ATM necessarily:

```
apply_for_loan()
print_statement()
```

as part of its required interface?

Maybe not.

The interface is too large.

18. ISP Solution

Break large interfaces into smaller, focused interfaces.

```
class Depositable(ABC):

    @abstractmethod
    def deposit(self, amount):
        pass
```

```
class Withdrawable(ABC):

    @abstractmethod
    def withdraw(self, amount):
        pass
```

```
class Transferable(ABC):
```



```
@abstractmethod
def transfer(self, amount):
    pass

class StatementProvider(ABC):

    @abstractmethod
    def generate_statement(self):
        pass
```

Now a class only implements what it actually supports.

19. Python and Multiple Interfaces

Python allows multiple inheritance, so a class can implement multiple capabilities:

```
class SavingsAccount(
    Depositable,
    Withdrawable,
    Transferable
):
    ...
```

While another account could implement only:

```
class FixedDepositAccount:
    ...
```

The important idea is:

Capability-based design instead of one giant interface.

D — Dependency Inversion Principle

20. What is DIP?

Dependency Inversion Principle:

High-level modules should not depend on low-level modules. Both should depend on abstractions.

And:

Abstractions should not depend on details. Details should depend on abstractions.

This is often the hardest SOLID principle initially.

21. What is Tight Coupling?

Consider:

```
class Bank:

    def __init__(self):
        self.database = MySQLDatabase()
```

Now `Bank` is directly coupled to:

`MySQLDatabase`

If we want PostgreSQL:

```
self.database = PostgreSQLDatabase()
```

we must modify the `Bank` class.

If we want MongoDB:

```
self.database = MongoDBDatabase()
```

again we modify `Bank`.

22. DIP Solution

Define an abstraction:

```
from abc import ABC, abstractmethod

class AccountRepository(ABC):

    @abstractmethod
    def save(self, account):
        pass

    @abstractmethod
    def find_by_id(self, account_number):
        pass
```

MySQL implementation:

```
class MySQLAccountRepository(AccountRepository):

    def save(self, account):
        print("Saving to MySQL")

    def find_by_id(self, account_number):
        print("Fetching from MySQL")
```

PostgreSQL implementation:

```
class PostgreSQLAccountRepository(AccountRepository):

    def save(self, account):
        print("Saving to PostgreSQL")

    def find_by_id(self, account_number):
        print("Fetching from PostgreSQL")
```

Now:

```
class Bank:

    def __init__(self, repository: AccountRepository):
        self.repository = repository
```

We can inject:

```
mysql_repository = MySQLAccountRepository()

bank = Bank(mysql_repository)
```

or:

```
postgres_repository = PostgreSQLAccountRepository()

bank = Bank(postgres_repository)
```

The `Bank` class doesn't care which database is being used.

This is **Dependency Injection**.

23. Dependency Inversion vs Dependency Injection

These terms are related but not identical.

Dependency Inversion Principle

is the design principle:

Depend on abstractions rather than concrete implementations.

Dependency Injection

is one technique used to implement that principle.

For example:

```
class Bank:

    def __init__(self, repository):
        self.repository = repository
```

The dependency is supplied from outside.

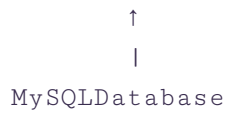
Therefore:

Without **DI**:

Bank → MySQLDatabase

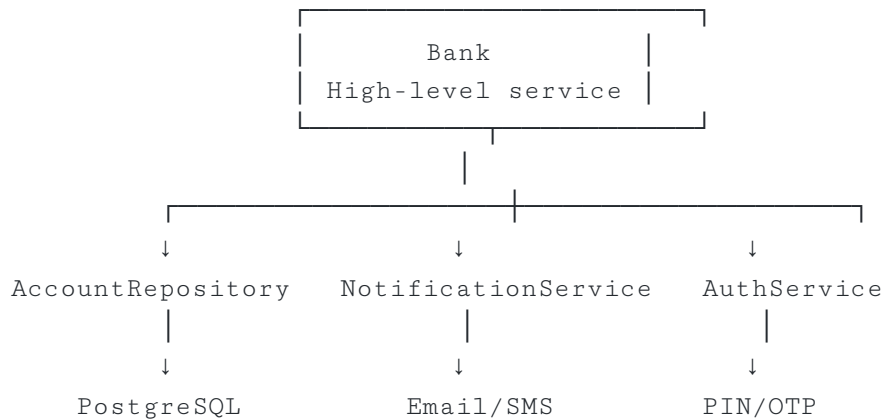
With DI:

Bank → AccountRepository

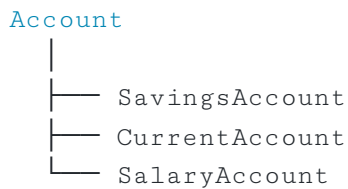


24. Putting All Five Together

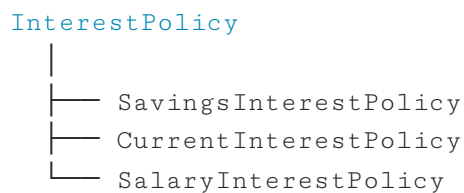
A good enterprise architecture might look like:



And:



with policies:



This architecture allows individual components to change independently.

25. SOLID and the Activity 3 Account

Let's revisit our Activity 3 class.

```
class Account:  
    ...
```

It currently handles:

```
Account  
├── identity  
├── age validation  
├── account type validation  
├── minimum balance  
├── deposits  
├── withdrawals  
├── PIN management  
├── account lifecycle  
└── status
```

This is perfectly reasonable for teaching basic OOP.

But as the banking application becomes an enterprise system, we can refactor it.

26. Enterprise Banking Architecture

We can divide the system into:

```
bank/  
├──  
├── domain/  
│   ├── account.py  
│   ├── policies.py  
│   └── interfaces.py  
├──  
├── services/  
│   └── account_service.py
```

```

|   └─ notification_service.py
|
└─ repositories/
    └─ account_repository.py
└─ main.py

```

For this demonstration, we can keep everything in one file first so students understand the design before introducing project structure.

27. SOLID Bank Implementation

```

from abc import ABC, abstractmethod
from dataclasses import dataclass

# =====
# DOMAIN
# =====

class Account(ABC):
    """
    Base abstraction for a bank account.

    Common account identity and lifecycle behavior lives here.
    Specific account rules are implemented by subclasses/policies.
    """

    def __init__(
        self,
        account_number: int,
        name: str,
        age: int,
        initial_balance: float
    ) -> None:

        self._account_number = account_number
        self._name = name
        self._age = max(age, 18)
        self._balance = max(initial_balance, self.minimum_balance)
        self._status = "Active"
        self._pin: int | None = None

    @property

```

```

@abstractmethod
def minimum_balance(self) -> float:
    pass

@property
def account_number(self) -> int:
    return self._account_number

@property
def name(self) -> str:
    return self._name

@property
def age(self) -> int:
    return self._age

@property
def balance(self) -> float:
    return self._balance

@property
def status(self) -> str:
    return self._status

def deposit(self, amount: float) -> bool:

    if self._status != "Active":
        return False

    if amount <= 0:
        return False

    self._balance += amount
    return True

def withdraw(
    self,
    amount: float,
    pin: int | None = None
) -> bool:

    if self._status != "Active":
        return False

    if self._pin is not None:
        if pin is None or pin != self._pin:
            return False

    if amount <= 0:

```



```

        return False

    if self._balance - amount < self.minimum_balance:
        return False

    self._balance -= amount
    return True

def close_account(self) -> bool:

    if self._status == "Inactive":
        return False

    self._status = "Inactive"
    return True

def reopen_account(self) -> bool:

    if self._status == "Active":
        return False

    self._status = "Active"
    return True

def set_pin(self, pin: int) -> bool:

    if 1000 <= pin <= 9999:
        self._pin = pin
        return True

    return False

def verify_pin(self, pin: int) -> bool:

    return self._pin is not None and self._pin == pin

def has_pin(self) -> bool:

    return self._pin is not None

```

28. Savings Account

```

class SavingsAccount(Account):

    @property

```

```
def minimum_balance(self) -> float:
    return 500.0
```

This class has a very specific responsibility:

Define Savings Account behavior.

29. Current Account

```
class CurrentAccount(Account):

    @property
    def minimum_balance(self) -> float:
        return 1000.0
```

Again, its responsibility is clear.

30. Interest Policies

Instead of putting interest logic inside `Account`, we create an abstraction.

```
class InterestPolicy(ABC):

    @abstractmethod
    def calculate(self, balance: float) -> float:
        pass
```

Savings:

```
class SavingsInterestPolicy(InterestPolicy):

    def calculate(self, balance: float) -> float:
        return balance * 0.04
```

Current:

```
class CurrentInterestPolicy(InterestPolicy):  
  
    def calculate(self, balance: float) -> float:  
        return balance * 0.01
```

Now the bank can introduce:

```
class SalaryInterestPolicy(InterestPolicy):  
  
    def calculate(self, balance: float) -> float:  
        return balance * 0.05
```

without modifying existing interest policies.

This demonstrates **OCP**.

31. Repository Abstraction

The Bank should not know whether data is stored in:

- MySQL
- PostgreSQL
- MongoDB
- SQLite
- Redis
- An external API

Create an abstraction:

```
class AccountRepository(ABC):  
  
    @abstractmethod  
    def save(self, account: Account) -> None:  
        pass  
  
    @abstractmethod  
    def find(self, account_number: int) -> Account | None:  
        pass
```

32. In-Memory Repository

For our classroom demonstration:

```
class InMemoryAccountRepository(AccountRepository):

    def __init__(self) -> None:
        self._accounts: dict[int, Account] = {}

    def save(self, account: Account) -> None:
        self._accounts[account.account_number] = account

    def find(self, account_number: int) -> Account | None:
        return self._accounts.get(account_number)
```

Notice something important.

`Bank` doesn't depend on this implementation.

It depends on:

```
AccountRepository
```

This demonstrates **DIP**.

33. Notification Interface

Instead of hardcoding:

```
send_email()
```

inside our `Bank`:

```
class NotificationService(ABC):

    @abstractmethod
    def send(self, recipient: str, message: str) -> None:
        pass
```

Email:

```
class EmailNotificationService(NotificationService):

    def send(self, recipient: str, message: str) -> None:
        print(f"EMAIL → {recipient}: {message}")
```

SMS:

```
class SMSNotificationService(NotificationService):

    def send(self, recipient: str, message: str) -> None:
        print(f"SMS → {recipient}: {message}")
```

WhatsApp:

```
class WhatsAppNotificationService(NotificationService):

    def send(self, recipient: str, message: str) -> None:
        print(f"WHATSAPP → {recipient}: {message}")
```

Adding WhatsApp does not require changing the Bank.

That is **OCP + DIP**.

34. Account Service

Now create a high-level service.

```
class Bank:

    def __init__(
        self,
        repository: AccountRepository,
        notification_service: NotificationService
    ) -> None:

        self._repository = repository
        self._notification_service = notification_service
```

Notice:

```
repository: AccountRepository
```

not:

```
repository: MySQLAccountRepository
```

and:

```
notification_service: NotificationService
```

not:

```
notification_service: EmailNotificationService
```

This is Dependency Inversion.

35. Creating Accounts

```
class Bank:

    def __init__(
        self,
        repository: AccountRepository,
        notification_service: NotificationService
    ) -> None:

        self._repository = repository
        self._notification_service = notification_service

    def create_savings_account(
        self,
        account_number: int,
        name: str,
        age: int,
        initial_balance: float
    ) -> Account:
```

```
account = SavingsAccount(  
    account_number,  
    name,  
    age,  
    initial_balance  
)  
  
self._repository.save(account)  
  
return account
```

Current account:

```
def create_current_account(  
    self,  
    account_number: int,  
    name: str,  
    age: int,  
    initial_balance: float  
) -> Account:  
  
    account = CurrentAccount(  
        account_number,  
        name,  
        age,  
        initial_balance  
    )  
  
    self._repository.save(account)  
  
    return account
```

36. Deposit Through Bank

```
def deposit(  
    self,  
    account_number: int,  
    amount: float  
) -> bool:  
  
    account = self._repository.find(account_number)  
  
    if account is None:
```

```

        return False

    success = account.deposit(amount)

    if success:
        self._notification_service.send(
            account.name,
            f"Deposited Rs. {amount}"
        )

    return success

```

Notice the Bank does not know:

- How accounts are stored
- How notification is sent
- Whether notification is Email/SMS/WhatsApp
- How Savings validates minimum balance

Each component has its own responsibility.

37. Withdrawal Through Bank

```

def withdraw(
    self,
    account_number: int,
    amount: float,
    pin: int | None = None
) -> bool:

    account = self._repository.find(account_number)

    if account is None:
        return False

    success = account.withdraw(amount, pin)

    if success:
        self._notification_service.send(
            account.name,
            f"Withdrawn Rs. {amount}"
        )

```



```
return success
```

Again:

```
Bank
↓
AccountRepository
↓
Account

Bank
↓
NotificationService
↓
Email/SMS/WhatsApp
```

38. Complete SOLID Bank Example

Here is the complete classroom implementation in one file.

```
from abc import ABC, abstractmethod

# =====
# ACCOUNT ABSTRACTION
# =====

class Account(ABC):

    def __init__(
        self,
        account_number: int,
        name: str,
        age: int,
        initial_balance: float
    ) -> None:

        self._account_number = account_number
        self._name = name
        self._age = max(age, 18)
        self._balance = max(initial_balance, self.minimum_balance)
```

```

        self._status = "Active"
        self._pin: int | None = None

    @property
    @abstractmethod
    def minimum_balance(self) -> float:
        pass

    @property
    def account_number(self) -> int:
        return self._account_number

    @property
    def name(self) -> str:
        return self._name

    @property
    def age(self) -> int:
        return self._age

    @property
    def balance(self) -> float:
        return self._balance

    @property
    def status(self) -> str:
        return self._status

    def deposit(self, amount: float) -> bool:

        if self._status != "Active":
            return False

        if amount <= 0:
            return False

        self._balance += amount
        return True

    def withdraw(
        self,
        amount: float,
        pin: int | None = None
    ) -> bool:

        if self._status != "Active":
            return False

        if self._pin is not None:

```

```

        if pin is None or pin != self._pin:
            return False

    if amount <= 0:
        return False

    if self._balance - amount < self.minimum_balance:
        return False

    self._balance -= amount
    return True

def close_account(self) -> bool:

    if self._status == "Inactive":
        return False

    self._status = "Inactive"
    return True

def reopen_account(self) -> bool:

    if self._status == "Active":
        return False

    self._status = "Active"
    return True

def set_pin(self, pin: int) -> bool:

    if 1000 <= pin <= 9999:
        self._pin = pin
        return True

    return False

def verify_pin(self, pin: int) -> bool:

    return self._pin is not None and self._pin == pin

def has_pin(self) -> bool:

    return self._pin is not None

```

```

# =====
# ACCOUNT TYPES
# =====

```

```

class SavingsAccount(Account):

    @property
    def minimum_balance(self) -> float:
        return 500.0

class CurrentAccount(Account):

    @property
    def minimum_balance(self) -> float:
        return 1000.0

# =====
# INTEREST POLICY
# =====

class InterestPolicy(ABC):

    @abstractmethod
    def calculate(self, balance: float) -> float:
        pass

class SavingsInterestPolicy(InterestPolicy):

    def calculate(self, balance: float) -> float:
        return balance * 0.04

class CurrentInterestPolicy(InterestPolicy):

    def calculate(self, balance: float) -> float:
        return balance * 0.01

# =====
# REPOSITORY
# =====

class AccountRepository(ABC):

    @abstractmethod
    def save(self, account: Account) -> None:
        pass

    @abstractmethod
    def find(self, account_number: int) -> Account | None:

```

```

        pass

class InMemoryAccountRepository(AccountRepository):

    def __init__(self) -> None:
        self._accounts: dict[int, Account] = {}

    def save(self, account: Account) -> None:
        self._accounts[account.account_number] = account

    def find(self, account_number: int) -> Account | None:
        return self._accounts.get(account_number)

# =====
# NOTIFICATION
# =====

class NotificationService(ABC):

    @abstractmethod
    def send(
        self,
        recipient: str,
        message: str
    ) -> None:
        pass

class EmailNotificationService(NotificationService):

    def send(
        self,
        recipient: str,
        message: str
    ) -> None:

        print(f"EMAIL → {recipient}: {message}")

class SMSNotificationService(NotificationService):

    def send(
        self,
        recipient: str,
        message: str
    ) -> None:

```

```
print(f"SMS → {recipient}: {message}")
```

```
# =====  
# BANK - HIGH LEVEL SERVICE  
# =====
```

```
class Bank:
```

```
    def __init__(  
        self,  
        repository: AccountRepository,  
        notification_service: NotificationService  
    ) -> None:  
  
        self._repository = repository  
        self._notification_service = notification_service
```

```
    def create_savings_account(  
        self,  
        account_number: int,  
        name: str,  
        age: int,  
        initial_balance: float  
    ) -> Account:  
  
        account = SavingsAccount(  
            account_number,  
            name,  
            age,  
            initial_balance  
        )  
  
        self._repository.save(account)  
  
        return account
```

```
    def create_current_account(  
        self,  
        account_number: int,  
        name: str,  
        age: int,  
        initial_balance: float  
    ) -> Account:  
  
        account = CurrentAccount(  
            account_number,  
            name,  
            age,
```

```

        initial_balance
    )

    self._repository.save(account)

    return account

def deposit(
    self,
    account_number: int,
    amount: float
) -> bool:

    account = self._repository.find(account_number)

    if account is None:
        return False

    success = account.deposit(amount)

    if success:
        self._notification_service.send(
            account.name,
            f"Deposited Rs. {amount}"
        )

    return success

def withdraw(
    self,
    account_number: int,
    amount: float,
    pin: int | None = None
) -> bool:

    account = self._repository.find(account_number)

    if account is None:
        return False

    success = account.withdraw(amount, pin)

    if success:
        self._notification_service.send(
            account.name,
            f"Withdrawn Rs. {amount}"
        )

```

```
return success
```

39. Using the SOLID Bank

```
repository = InMemoryAccountRepository()

notification = EmailNotificationService()

bank = Bank(
    repository,
    notification
)
```

Create an account:

```
account = bank.create_savings_account(
    101,
    "Ravi",
    17,
    200
)
```

The account automatically corrects:

```
Age:
17 → 18

Balance:
Rs. 200 → Rs. 500
```

Set PIN:

```
account.set_pin(1234)
```

Deposit:

```
bank.deposit(101, 1000)
```


Withdraw:

```
bank.withdraw(101, 500, 1234)
```

Wrong PIN:

```
bank.withdraw(101, 500, 9999)
```

Result:

```
False
```

40. Changing Email to SMS

Initially:

```
notification = EmailNotificationService()  
  
bank = Bank(  
    repository,  
    notification  
)
```

Later:

```
notification = SMSNotificationService()  
  
bank = Bank(  
    repository,  
    notification  
)
```

We did **not modify** the `Bank` class.

This demonstrates:

- OCP
- DIP

41. Adding a New Database

Suppose the application moves to PostgreSQL.

We create:

```
class PostgreSQLAccountRepository(AccountRepository):  
  
    def save(self, account: Account) -> None:  
        print("Saving account to PostgreSQL")  
  
    def find(self, account_number: int) -> Account | None:  
        print("Searching PostgreSQL")  
        return None
```

Then:

```
repository = PostgreSQLAccountRepository()  
  
bank = Bank(  
    repository,  
    notification  
)
```

Again:

```
Bank did not change.
```

That is exactly what we want in an enterprise system.

42. Where Each SOLID Principle Appears

Principle	Our Bank Implementation
S – SRP	Account, repository, notification and interest logic are separated
O – OCP	New account types, interest policies and notification systems can be added
L – LSP	Savings/Current accounts can be used wherever <code>Account</code> is expected

Principle	Our Bank Implementation
I – ISP	Small focused abstractions such as <code>InterestPolicy</code> , <code>AccountRepository</code> , <code>NotificationService</code>
D – DIP	<code>Bank</code> depends on repository/notification abstractions instead of concrete implementations

43. The Most Important Enterprise-Level Concept

Students should understand that SOLID is **not about writing more classes**.

Bad interpretation:

```

One  class
  ↓
Five classes
  ↓
Twenty classes
  ↓
SOLID achieved!

```

That is incorrect.

The actual objective is:

```

Low Coupling
  +
High Cohesion
  +
Clear Responsibilities
  +
Extensibility
  +
Testability

```

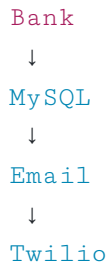
44. Coupling vs Cohesion

These two terms are extremely important for enterprise development.

Coupling

Coupling means how strongly one component depends on another.

Bad:



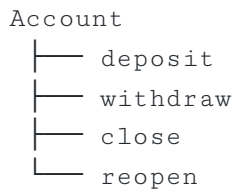
Everything is directly connected.

High coupling makes change difficult.

Cohesion

Cohesion means how closely related the responsibilities inside a class are.

Good:



These operations are closely related.

Therefore, Account has high cohesion.

45. SOLID Does Not Mean "Never Modify Code"

This is another important misconception.

OCP does not literally mean:

"Never modify existing code."

It means:

Design the system so that frequently changing behavior can be extended without unnecessarily modifying stable components.

For example, adding:

```
class WhatsAppNotificationService(NotificationService):
```

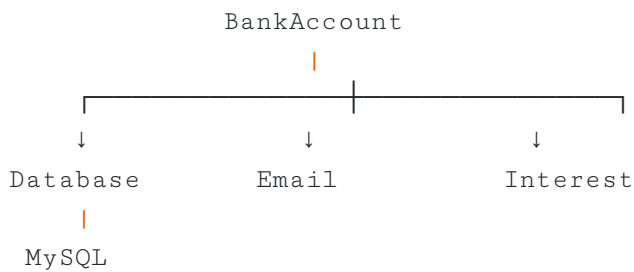
should not require modifying:

```
Bank
```

because notification behavior is designed for extension.

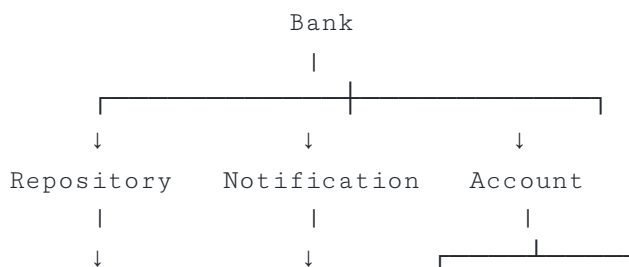
46. SOLID Before and After

Before



One class knows too much.

After



PostgreSQL Email Savings Current
SMS
WhatsApp

Each component has a focused responsibility.

47. Trainer Demonstration: A Change Request

Give students this scenario:

Business Requirement: The bank currently sends transaction notifications through email. Management now wants SMS notifications as well.

Poor Design

```
class Bank:

    def withdraw(self, account, amount):

        account.withdraw(amount)

        send_email(
            account.name,
            "Withdrawal successful"
        )
```

To add SMS:

```
class Bank:

    def withdraw(self, account, amount):

        account.withdraw(amount)

        send_email(...)
        send_sms(...)
```

We modified the Bank.

SOLID Design

```
class NotificationService(ABC):  
  
    @abstractmethod  
    def send(self, recipient, message):  
        pass
```

Then:

```
class SMSNotificationService(NotificationService):  
  
    def send(self, recipient, message):  
        print("Sending SMS")
```

And:

```
bank = Bank(  
    repository,  
    SMSNotificationService()  
)
```

The Bank remains unchanged.

48. A Practical Mental Model for Students

When designing a class, ask:

S – Single Responsibility

"What is this class responsible for?"

If the answer is:

"It manages accounts, sends emails, saves databases and generates reports."

You probably have a problem.

O – Open/Closed

Ask:

"What happens when the business adds a new type?"

If adding a new account requires modifying a huge `if/elif` chain everywhere, consider an abstraction or strategy.

L — Liskov

Ask:

"Can I replace the parent object with this child without breaking the program?"

If not, inheritance may be inappropriate.

I — Interface Segregation

Ask:

"Does this class really need every method in this interface?"

If not, split the interface.

D — Dependency Inversion

Ask:

"Am I depending directly on a concrete implementation?"

Instead of:

```
Bank → MySQLDatabase
```

prefer:

```
Bank
↓
Database Interface
↑
MySQL / PostgreSQL / MongoDB
```


49. SOLID Is a Design Decision, Not a Syntax Feature

Students should understand that Python does not have a:

```
solid=True
```

keyword.

SOLID is about **how we structure our code**.

Python gives us tools that help implement SOLID:

```
class
```

for objects,

```
ABC
```

for abstractions,

```
abstractmethod
```

for contracts,

```
@property
```

for encapsulation,

and **dependency injection** through constructors:

```
Bank(repository, notification_service)
```

50. Final Takeaway

The progression of our banking activities can now be explained as:

```
Activity 1
  ↓
Basic Python

Activity 2
  ↓
Classes & Objects

Activity 3
  ↓
Validation + State + Security

SOLID
  ↓
Designing maintainable classes

Enterprise Architecture
  ↓
Services + Repositories + Interfaces
  ↓
Scalable Applications
```

OOB teaches us how to create objects.

SOLID teaches us how to organize those objects into systems that can survive change.

And ultimately:

```
Good Code
  ↓
Works

SOLID Code
  ↓
Works
  +
Can Change
  +
Can Scale
  +
Can Be Tested
  +
Can Be Maintained
```

This is the transition from “I know Python classes” to “I can reason about how enterprise software is designed.”